FinerDedup: Sifting Fingerprints for Efficient Data Deduplication on Mobile Devices

Xianzhang Chen, Xingjie Zhou, Wei Li*, Xi Yu, Duo Liu, Yujuan Tan, Ao Ren
College of Computer Science, Chongqing University, Chongqing, China
{xzchen,yuxi,liuduo,tanyujuan,ren.ao}@cqu.edu.cn,{zhouxingjie,liwei}@stu.cqu.edu.cn

Abstract

Data deduplication is promised to extend the lifetime and capacity of storage on mobile devices. However, existing data deduplication works show high memory consumption and indexing costs for maintaining a fingerprint for each data block, especially when the duplicate ratio of data blocks on mobile systems is about 10% to 30%. In this paper, we propose a novel approach called FinerDedup to optimize the memory costs and retrieval efficiency of data deduplication. FinerDedup drastically reduces the number of fingerprints by screening out the duplicate data blocks via random forest and Bloom filter. We implement FinerDedup on real mobile devices with Android 10 and evaluate it with real workloads. Extensive experimental results show that FinerDedup can reduce 85% of fingerprints and 20% of I/O latency over the widely-used DmDedup.

Keywords

Data deduplication, machine learning, mobile systems

ACM Reference Format:
Xianzhang Chen, Xingjie Zhou, Wei Li*, Xi Yu, Duo Liu, Yujuan Tan, Ao Ren. 2024. FinerDedup: Sifting Fingerprints for Efficient Data Deduplication on Mobile Devices. In *61st ACM/IEEE Design Automation Conference (DAC '24), June 23–27, 2024, San Francisco, CA, USA*. ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3649329.3657307

1 Introduction

With the ever-increasing storage consumption of mobile applications, both users and manufactures of mobile devices are bothered by the limited storage space on the devices [4, 9]. In this case, data deduplication is promised to extend the storage limitations of mobile devices by saving the storage space of duplicated data [11–13, 16]. The mainstream manufactures of mobile devices, such as Apple [5] and Huawei [6], are trying to provide data deduplication services for users.

There have been several data deduplication approaches for mobile systems in the past decade. DmDedup [14] is one of the most widely-used data deduplication mechanisms, which builds a fingerprint for each data block by hashing and avoids redundant writes of duplicate data blocks by retrieving the fingerprint table. SmartDedup [16] uses online deduplication in the foreground to reduce redundant writing and runs offline deduplication in the background to find omissions. APP-Dedupe [12] divides the entire fingerprints into different groups and only maintains the fingerprints of the foreground application in the memory. In general, existing approaches rely on a huge fingerprint table that precisely records a fingerprint for each data block to find out the duplicate data blocks.

Unfortunately, the overall duplication ratio of data on mobile systems is merely about 10% to 30% [12, 16]. It means that about 70% to 90% of the fingerprints in the fingerprint table of existing data deduplication systems are useless. These useless fingerprints not only bring large memory footprints but also degrade performance for loading and retrieving them before data accesses. Therefore, suppose we can find out the duplicate data blocks and build a fingerprint table only for them. We can drastically reduce the overhead for maintaining and searching the fingerprints of data blocks.

In this paper, we propose an efficient and low-overhead data deduplication method, namely FinerDedup, for mobile systems by cutting off the useless fingerprints. FinerDedup recognizes data blocks as two types: 1) *unique* data blocks that are unlikely to be duplicated in the future and 2) *duplicate* data blocks that will be duplicated in the future. Given a new write request, FinerDedup predicts the type of the corresponding data block using a pre-trained random forest [3], which is a lightweight classification model that consumes minimal performance on mobile devices. To prepare the model, FinerDedup collects the features of writes, transforms them into datasets, and trains the random forest when the system is idle.

Moreover, FinerDedup designs a Bloom filter [2, 8] along with the fingerprint table to correct the possible false predictions. The Bloom filter can filter out the duplicate data blocks that are predicted to be unique. Then, FinerDedup will save fingerprints for these data blocks in the fingerprint table. The counters in the fingerprint table are used to periodically remove those fingerprints of the unique data blocks that are predicted to be duplicated. Finally, FinerDedup uses Linux kernel's device mapper framework to formulate a mapping strategy. Compared with other deduplication systems, FinerDedup can keep fewer fingerprints and achieve lower I/O latency.

We implement FinerDedup on the Google Pixel 3 smartphones. We evaluate the performance and overhead of FinerDedup at different duplication rates using typical benchmark, Flexible I/O (FIO) [1], and real-world applications. The experimental results show that FinerDedup outperforms DmDedup [14] when the duplication rate is lower than 30%. In real workloads, compared with DmDedup, we can achieve more than 92% deduplication and reduce more than 85% of fingerprints. In summary, FinerDedup can minimize the overhead caused by useless fingerprints while ensuring efficient data deduplication.

*Wei Li is the corresponding author.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

ACM ISBN 979-8-4007-0601-1/24/06
https://doi.org/10.1145/3649329.3657307

The main contributions of this paper include:

- We propose a feature-based data deduplication method for mobile devices, FinerDedup, for saving memory footprints and reducing I/O latency.
- We implement Finerdedup on a real mobile system running with Android.
- Extensive experimental results show that FinerDedup significantly reduces memory costs and improves write performance over the existing data deduplication approach.

2 Background and Motivation

2.1 Data Duplicates in Mobile Systems

The ability of data deduplication technologies has drawn significant attention from the manufacturers of mobile devices in extending the storage capacity of mobile systems and thereby enhancing user engagement. For example, the smartphones using HarmonyOS of Huawei have already supported data deduplication [6]. Before deploying a data deduplication mechanism on mobile devices, it is essential to understand the features of data on mobile systems.

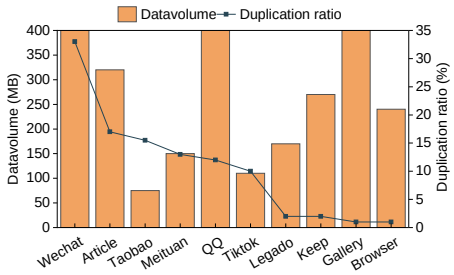


Figure 1: Application duplication ratio of mobile device

To this end, we collect user data from real-world applications to ascertain data duplication on mobile devices. Figure 1 displays the data duplication ratios of typical mobile applications with varied data volumes (a mere 400M data points are visible, and applications like WeChat and QQ have even larger volumes). In our experiments, the majority of duplication rates are below 20%, averaging 12.27%. Existing works, such as App-dedup [12] and SmartDedup [16] introduce that the overall duplication rate of mobile device applications is about 20% to 33%. Consequently, we estimate that the data duplication ratio of mobile devices is approximately 10% to 30%, which is significantly lower than the data load in traditional deduplication scenarios, such as backup systems. Hence, suppose the mobile applications store 100GB data on a mobile system. It may contain 10GB to 30GB of duplicate data. Due to the highly limited capacity, generally smaller than 256GB, and lifespan of mobile devices, it is meaningful for users to have tens of gigabytes more of storage space, taking advantage of data deduplication techniques.

2.2 High Costs of Existing Data Deduplication

There are many data deduplication technologies available for mobile systems [7, 10, 15, 17, 18]. For example, the widely-used DmDedup [14] is an efficient block-level deduplication solution for Linux-based systems, such as Android [13]. DmDedup saves the fingerprints of all data blocks to ensure that every write instance coming to DmDedup is deduplicated against the previously written data. Similarly, SmartDedup [16] also maintains the fingerprints of all data blocks. Differently, SmartDedup uses a two-level architecture to reduce the run-time memory footprints of fingerprints by caching part of the fingerprints in the memory and storing the whole fingerprint table in the storage.

Nevertheless, existing data deduplication methods cause a high overhead for managing the useless fingerprints of unique data in mobile systems with low duplication ratios. Taking DmDedup as an example, when writing 100GB of data that the deduplication ratio is 20%, DmDedup has to maintain a 540MB fingerprint table where each entry of a data block is 24B for all the data blocks. In the fingerprint table, the size of fingerprints of unique data blocks is 480MB, which will not be used in the future. As a result, it is inevitable to cause high overhead for data deduplication with such a big table in mobile systems by either maintaining the whole table in the memory or swapping the required fingerprints from the storage. We believe that a promising low-overhead data deduplication for mobile systems with a low duplication ratio is to cut off the overhead of the useless fingerprints of unique data.

3 FinerDedup Design

3.1 Overview of FinerDedup

The main idea of FinerDedup is to reduce the costs of fingerprints by predicting and sifting out the unique data blocks that will never appear again in write operations. Specifically, FinerDedup uses the random forest [3] to predict the potential labels (i.e., *duplicate* or *unique*) of data blocks and decides whether to write the data blocks and save their corresponding fingerprints. Figure 2 shows the architecture of FinerDedup, which consists of four major components: *Data block classifier*, *Bloom filter*, *block address mapper*, and *fingerprint table in main memory*.

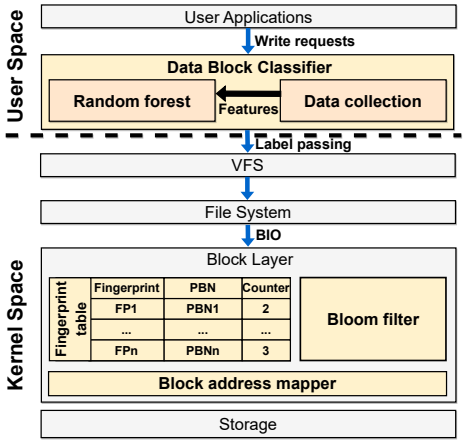


Figure 2: Architectural view of FinerDedup

Data block classifier. FinerDedup classifies the incoming data blocks into two categories, i.e., duplicate data blocks and unique data blocks, using the data block classifier. The data block classifier

takes part of the metadata attributes of a data block and its associated directory as features. Based on these features, we collect daily write data from users for incrementally training the model. The training of the model of the data block classifier only occurs when the device is idle. During runtime, when receiving a write request from a user application, the classifier first collects the feature of the corresponding data block in *libc*. Then, the classifier predicts whether the data block is a potential duplicate one. Finally, the data block classifier transmits the predicted label of the requested data block to the block layer via a critical write path.

Bloom filter. Similar to other learning models, the model in the data block classifier may make false predictions, especially for the data blocks marked as *unique* by the model but actually are duplicate ones. FinerDedup handles such false predictions using a dedicated Bloom filter [2, 8]. The Bloom filter is a space-efficient probabilistic data structure used to test whether an element is a set member. It mixes a long binary vector and a series of random hash functions. The Bloom filter receives the fingerprints of data blocks with *unique* labels as input to check whether these fingerprints have appeared in the past.

Block address mapper. Situated at the device mapper layer, the block address mapper eliminates the need to write data blocks with identical fingerprints into the storage. FinerDedup modifies the mapping from the logical block numbers (LBNs) to the physical block numbers (PBNs) to establish a mapping where multiple logical blocks with the same fingerprint are associated with the same physical block.

Fingerprint table in main memory. The fingerprint table is used to store and retrieve the hash values, i.e., fingerprints, associated with the stored data blocks. Additionally, it adds the counters for handling *False positive* predictions and maintains the Hash-PBN mapping, which represents the relationship between the fingerprints of data blocks and their corresponding physical block numbers. FinerDedup relies on querying and comparing the fingerprints of the blocks to identify identical data. The fingerprint table empowers FinerDedup to determine whether the content of a data block has appeared before or is appearing for the first time.

3.2 Feature-based Data Deduplication

In this section, we introduce the methods for collecting training data, selecting features, and labeling data, as well as the training of machine learning models and the metrics for analyzing the model.

Data collection. The initial training data for the data block classifier is sourced from the daily application usage of device users. We capture the features of write data blocks from all applications and calculate their fingerprints. With a large number of applications on mobile devices that evolve quickly, various apps may exhibit unique user behavior patterns. Therefore, customizing models for particular applications is costly and unnecessary. We modify the I/O critical path to support tracing users' daily data writing. We save the captured data features and fingerprints in the form of files.

Feature extracting and data labeling. After exploring various input features, we have determined to use the metadata attributes of data blocks and their respective directories as training features. Considering the limited resources of mobile devices, the number of features should not be excessive. Additionally, not all features are essential for model prediction, so we use Recursive Feature Elimination (RFE) to help us filter features. During our design iterations, we identify the three most essential features, which are: *d_ino* (directory inode number), *who* (owner of the file), and *pos* (current writing position offset in a file). All the features mentioned above are associated with user behavior. The accuracy can be improved by adding features about *file_mode* (permissions mode for a file or directory), *dentry_flags* (flags indicating dentry attributes), and *open_flag* (flags for file open operations). Furthermore, we traverse the fingerprints in the file to determine whether the label for each block is *duplicate* or *unique*. According to our tests, it takes about 1 minute to label 4GB of user write data. Finally, using the aforementioned features and labels can enable the random forest to achieve an overall accuracy rate of over 92%.

Machine learning based categorization. Considering the limited resource constraints on mobile devices, we need a high-precision categorization model with acceptable overhead. After evaluating a multitude of machine learning models, the data block classifier uses random forest as the categorization model to learn the input features. A random forest is a combination of multiple decision trees that harnesses the capabilities of multi-class decision trees for decision-making purposes. The model's training typically occurs during the device's idle time, so it does not affect normal user operations. We incrementally train and update the model with user data collected daily to ensure the model adapts to the users' daily behavior patterns. According to our tests, training with 4GB write data takes about 2 minutes on average, which is acceptable.

To avoid unnecessary overhead caused by extensive feature passing, we modify the critical write path to only pass the labels to the block layer. When subsequent write requests occur, we use the trained model to predict the labels of write data blocks. The transmission of data labels consists of two parts: from the user space to the kernel file system and from the kernel file system to the block layer. In user space, we modify the *libc write* function to pass the labels through a system call to the kernel and write them into the cache. The file system dispatches a *BIO* structure to the block layer, through which we convey the labels to the block layer.

Metrics for analyzing categorization model. The categorization model marks the data blocks as two types: *unique* or *duplicate*. In FinerDedup, we label data as *unique* if there is a greater probability that it will not appear again in the future. Otherwise, we label it as *duplicate* data. Unfortunately, the actual properties of the data blocks may be different. Hence, we divide the data blocks into four situations, as shown in Figure 3.

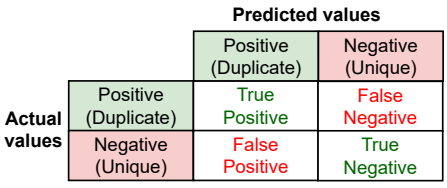


Figure 3: Confusion matrix

The true positive (TP) data blocks are marked as *duplicate* and will be written again in the future. The true negative (TN) data

blocks marked as *unique* and will not be written in the future. For both TP and TN data blocks, it is correct for us to record or discard the fingerprints of the data blocks. The false positive (FP) data blocks are marked as *duplicate* and will never be written in the future. FinerDedup erroneously records their fingerprints, which offers no benefits for future deduplication. Additionally, the increase in the fingerprint table affects the overall effectiveness of the entire deduplication process. We need to eliminate the redundant fingerprints caused by the FP data blocks. The false negative (FN) data blocks are marked as *unique* and will be written again in the future. FinerDedup incorrectly discards their fingerprints, leading to data blocks being redundantly written multiple times. Thus, we need a method to find out the FN data blocks to save storage space.

3.3 Handling False Predictions

As Section 3.2 mentions, there are two types of false predictions of data blocks: FP and FN. To minimize the cost of these two types of false predictions, FinerDedup uses the *counters* and the *Bloom filter* to handle the false predictions properly.

The Counters for handling false positive predictions. In the case of FP predictions, the categorization model of FinerDedup mistakenly regards the *unique* data blocks as duplicate data. As a result, the fingerprint table will consume more memory, and the retrieval time of fingerprints will be longer. To reduce the memory overhead caused by FP predictions, as shown in Figure 4, FinerDedup sets a reference counter for each fingerprint entry in the fingerprint table. This counter indicates whether the PBN (physical block numbers) has been shared. We modify the LBN-PBN mapping in the block address mapper to enable duplicate data blocks to share the same physical block. When a *duplicate* data block arrives, FinerDedup searches for the corresponding entry in the fingerprint table based on its fingerprint and then increments the entry's counter by 1. FinerDedup creates a new fingerprint entry and initializes the counter to 0 when a data block arrives for the first time. When a data block is modified, FinerDedup decrements the counter of its corresponding fingerprint entry by 1. Finally, FinerDedup sets a clearing period for the fingerprint table, regularly clearing entries with counter values 0. The duration of this period can be aligned with the frequency of model updates, such as one day.

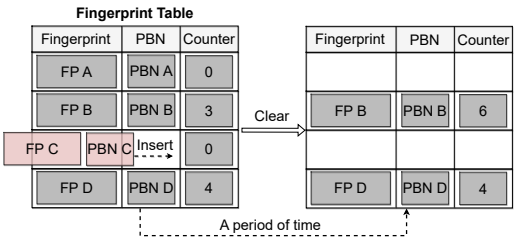


Figure 4: FP prediction handling

We provide an example to illustrate the function of the counter, as shown in Figure 4. ① The categorization model inaccurately predicts the *unique* data block associated with fingerprint C as *duplicate* data. Because this data block appears for the first time, FinerDedup inserts fingerprint C into an available entry in the fingerprint table. ② Next, the fingerprint table sets the clearing period to one day. ③ For fingerprints A and C, if their counter values remain at 0 after undergoing a clearing period, FinerDedup will clear their fingerprint entries.

A Bloom filter for handling false negative predictions. For the FN predictions, the categorization model of FinerDedup mistakenly marks the duplicate data as *unique*. The fingerprint table will not record the fingerprints of this type of data block, leading to redundant writing of duplicate data. Redundant writing exacerbates IO latency, consumes storage space, and diminishes system efficiency. FinerDedup uses the Bloom filter to identify FN predictions. The Bloom filter ascertains the presence of an input element through a binary vector and a series of hash functions. The Bloom filter takes as input the fingerprint of the data block labeled *unique* and maps it to certain bit positions in the vector through some hash functions. If all the bit positions value 1, it indicates that the *unique* data block has been previously saved, implying an erroneous prediction by the categorization model. Then, we consider to save the fingerprint for the data block. If the values at some bit positions are 0, indicating that the *unique* data block appears for the first time, then the Bloom filter needs to change these bit values to 1.

Bloom filter can accurately determine that a certain element is not in the set. However, there is a certain false positive rate when it determines an element is in the set, meaning elements not in the set are mistakenly identified as being in the set. This false positive rate depends on the size of the Bloom filter, the volume of elements input, and the quantity of hash functions. We can reduce the false positive rate by adjusting the size of the Bloom filter and the quantity of hash functions. Even if a misjudgment occurs, the only cost is storing an extra fingerprint.

3.4 Write Workflow

When FinerDedup receives a write request, as illustrated in Figure 5, it executes the following actions:

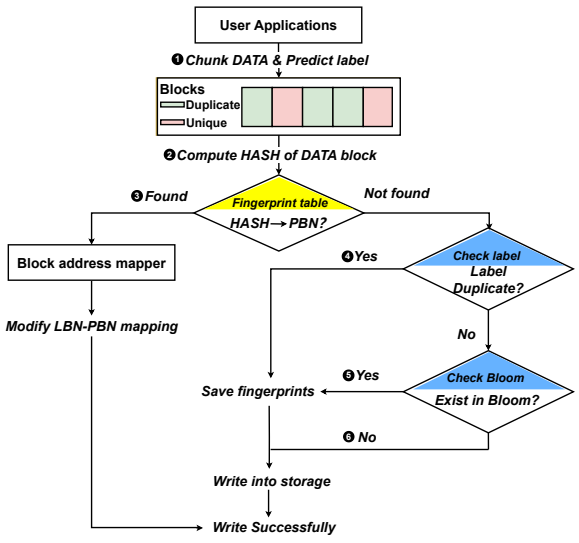


Figure 5: Write workflow of FinerDedup

❶ Upon an application sending a write request in user space, we collect the features of each data block. Using the categorization model, we obtain the labels of the data blocks. Next, we actively pass the label results to the block layer via the critical path of data writing. ❷ After the write data blocks arrive at the block layer, we compute their fingerprints and retrieve them in the fingerprint table. ❸ For data blocks with fingerprints already recorded in the fingerprint table, we only need to obtain their physical addresses from the table entries and establish the mapping relationship from logical addresses to physical addresses in the block address mapper. These types of data blocks will not continue to be written to storage. ❹ For data blocks with fingerprints not recorded in the fingerprint table, if the data block label is a *duplicate*, we save its fingerprint. ❺ If not, we check the Bloom filter to see whether the fingerprint has appeared previously. For fingerprints found in the Bloom filter, we save them in the fingerprint table and then modify the bit values in the Bloom filter. ❻ For fingerprints not found in the Bloom filter, we do not save their fingerprints. Finally, for step ❹, ❺ and ❻, we write the blocks into storage.

4 Evalution

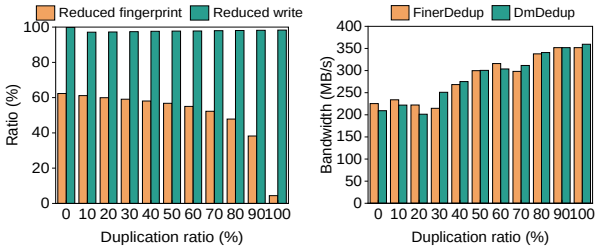
4.1 Environment Setup

To evaluate the effectiveness of FinerDedup, we implement FinerDedup on Google Pixel 3 smartphones composed of SDM 845, 4GB RAM, and 64GB flash storage. FinerDedup runs on Android 10 with Linux kernel version 4.9. We deploy FinerDedup and DmDedup [14] on these devices for evaluation. To comprehensively present the characteristics of FinerDedup, we compare the performance of FinerDedup to DmDedup, which is a traditional data deduplication method. We first evaluate and analyze the performance of FinerDedup with micro-benchmarks. Then, we evaluate the performance breakdown of FinerDedup. Finally, we run real-world applications on the smartphone to evaluate the performance of FinerDedup.

4.2 Micro-benchmarks

Utilizing FIO, we generate datasets with duplicate distributions ranging from 0% to 100%, divided into 11 distinct datasets. Each dataset comprises 300,000 blocks, each 4KB in size. As shown in Figure 6 (a), *Reduced fingerprint* represents the ratio of the difference in the number of fingerprints saved by FinerDedup and DmDedup to the number of fingerprints saved by DmDedup. *Reduced write* represents the ratio of the number of writes reduced by FinerDedup to the number of writes reduced by DmDedup. Regarding the volume of deduplicated data, Finerdedup reduces at least 50% of fingerprints compared to DmDedup while achieving at least 98% reductions in write operations. The reason for not achieving 100% reduced write is that DmDedup keeps the fingerprints of all write data, ensuring its reduced write rate is always 100%. In contrast, FinerDedup may have FP predictions, leading to a small amount of redundant writing.

As shown in Figure 6 (b), regarding write bandwidth, Finerdedup is superior to Dmdedup when the duplication rate is below 30%. Considering the common 20% data duplication rate on mobile devices, Finerdedup emerges as a superior option.

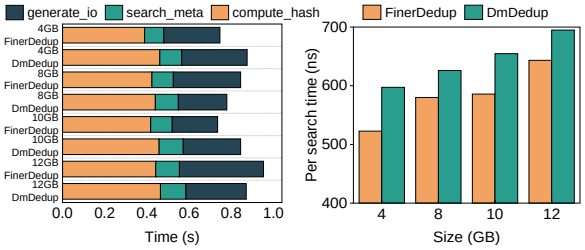


(a) Reduced fingerprints and writes (b) Bandwidth

Figure 6: Deduplication ratio and bandwidth using FIO.

4.3 Performance Breakdown

Figure 7 (a) illustrates the write latency of FinerDedup under various conditions. We write data sizes of 4GB, 8GB, 10GB, and 12GB to both systems to expand the fingerprints in memory. Subsequently, we calculate the time allocation for each phase.



(a) Write latency (b) Search time

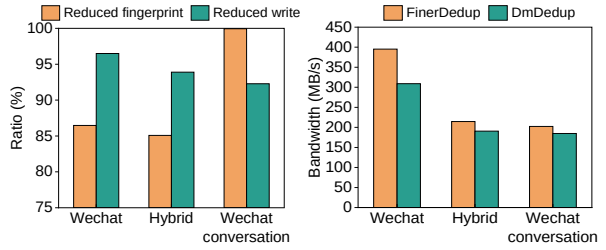
Figure 7: Breakdown of write latency and search time.

Figure 7(b) shows that the search time for each block in FinerDedup demonstrates a reduction over that in DmDedup, ranging between 50 and 70 ns. The main reason for the reduced search time is that FinerDedup maintains fewer fingerprints in memory, thereby enhancing retrieval efficiency. Despite the increase in data writing volume, the total search time increases only slightly because FinerDedup uses hash tables in memory to speed up indexing.

4.4 Real-world Application

We collect real application write data from users for analysis. As shown in Figure 8, we analyze three datasets: WeChat application write data, hybrid applications write data, and WeChat conversation write data. Furthermore, their data duplication rates are 28.2%, 7.8%, and 0.6%, respectively.

Figure 8 (a) shows the ratio of fingerprints and writes reduced by FinerDedup compared to DmDedup. The calculation methods for *Reduced fingerprint* and *Reduced write* are the same as those in Section 4.2. FinerDedup achieves a fingerprint saving rate of over 85% in real applications. The fingerprint saving rate for WeChat conversation is nearly 100% due to a high proportion of *unique* data blocks and a high recall rate of the model in predicting *unique* data blocks, resulting in relatively fewer fingerprints saved. Moreover, FinerDedup has attained over 90% in the reduced write operations



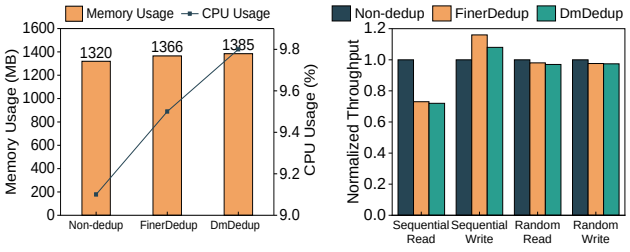
(a) Reduced fingerprints and writes (b) Bandwidth

Figure 8: Comparison of deduplication ratio and bandwidth using real-world applications.

ratio. Figure (b) shows that in scenarios with low data duplication rates on mobile devices, Finerdedup outperforms Dmdedup in write bandwidth across all worksets.

4.5 Overhead Analysis

System overhead. We set up a non-dedup device to compare the system overhead after equipping it with FinerDedup and DmDedup, respectively. The non-dedup device, a bare machine without data deduplication systems, uses the same configuration as in the experimental setup. We monitor the system CPU utilization using the *iostat* command and track the system memory usage with the *dumpsys* command. As shown in Figure 9 (a), the results indicate that, compared to the non-dedup device, FinerDedup incurs minimal memory overhead, not exceeding 3.4%, with an average of 2.2%. It occurs because FinerDedup selectively loads some of the data's fingerprints into memory. FinerDedup imposes minimal extra processing overhead on CPU utilization compared to the prototype system, with an average not exceeding 3.3%. Given the low system overhead, IO data deduplication could be a viable option for Android-based smartphones.



(a) Memory and CPU overhead (b) The normalized system throughput

Figure 9: Overhead analysis

Performance overhead. Figure 9 (b) presents the normalized system throughput under various conditions, contrasting the performance of the prototype system, FinerDedup, and Dmdedup. FinerDedup enhances sequential write throughput by 11.5% but reduces sequential read throughput by 30.0%. It illustrates the prevalent issue of read amplification in the data deduplication field. Data deduplication technology typically rearranges the layout of original data blocks to facilitate data sharing and conserve storage space. However, such rearrangement may result in dispersed data distribution across the storage space, consequently elevating the cost of read operations.

5 Conclusion

In this paper, we proposed a novel approach called FinerDedup to optimize the memory costs and retrieval efficiency of data deduplication. FinerDedup largely reduced the fingerprints by ignoring the predicted non-duplicate data blocks. We implemented and evaluated FinerDedup on real mobile systems. The experimental results proved that FinerDedup can effectively reduce the overhead of data deduplication on mobile systems.

Acknowledgments

This work is partially supported by Natural Science Foundation of China (Project No.62072059, 62102051, and 62372073), the Fundamental Research Funds for the Central Universities under Grant (Project No.2023CDJXY-039), and Chongqing Post-doctoral Science Foundation (Project No.2021LY75 and cx2023080). We would like to thank the anonymous reviewers for their valuable comments and improvements to this paper.

References

[1] 2017. *Fio: flexible i/o tester.* https://fio.readthedocs.io/en/latest/fio_doc.html
[2] Burton H. Bloom. 1970. Space/Time Trade-Offs in Hash Coding with Allowable Errors. 13, 7 (1970).
[3] L. Breiman. 2001. Random Forests. *Machine Learning* 45 (2001), 5–32.
[4] Aaron Carroll and Gernot Heiser. 2010. An analysis of power consumption in a smartphone. In *ATC*.
[5] Apple Inc. 2023. *Merge duplicate photos and videos on iPhone.* https://support.apple.com/guide/iphone/merge-duplicate-photos-and-videos-iph1978d9c23/ios
[6] Huawei Inc. 2023. *Clean up phone storage.* https://consumer.huawei.com/en/support/content/en-us15921348/
[7] Mark Lillibridge, Kave Eshghi, and Deepavali Bhagwat. 2013. Improving restore speed for backup systems that use inline chunk-based deduplication. In *FAST*. 183–197.
[8] Guanlin Lu, Young Jin Nam, and David HC Du. 2012. BloomStore: Bloom-filter based memory-efficient key-value store for indexing of data deduplication on flash. In *MSST*. 1–11.
[9] Aqeel Mahesri and Vibhore Vardhan. 2004. Power consumption breakdown on a modern laptop. In *PACS*. Springer, 165–180.
[10] N. Mandagere, P. Zhou, and M. A. Smith. 2008. Demystifying Data Deduplication. In *The ACM/IFIP/USENIX Middleware Conference Companion*. 12–17.
[11] Sonam Mandal, Geoff Kuenning, Dongju Ok, Varun Shastry, Philip Shilane, Sun Zhen, Vasily Tarasov, and Erez Zadok. 2016. Using hints to improve inline block-layer deduplication. In *FAST*. 315–322.
[12] Bo Mao, Jindong Zhou, Suzhen Wu, Hong Jiang, Xiao Chen, and Weijian Yang. 2019. Improving Flash Memory Performance and Reliability for Smartphones With I/O Deduplication. *IEEE TCAD* 38, 6 (2019), 1017–1027.
[13] Chunlin Song, Xianzhang Chen, Duo Liu, Xiaoliu Feng, Xi Yu, Jiali Li, Yujuan Tan, and Ao Ren. 2022. CADedup: High-performance Consistency-aware Deduplication Based on Persistent Memory. In *ICCD*. 726–729.
[14] Vasily Tarasov, Deepak Jain, Geoff Kuenning, Sonam Mandal, Karthikeyani Palanisami, Philip Shilane, Sagar Trehan, and Erez Zadok. 2014. Dmdedup: Device mapper target for data deduplication. In *2014 Ottawa Linux Symposium*. Citeseer.
[15] Wen Xia, Hong Jiang, Dan Feng, Fred Douglis, Philip Shilane, Yu Hua, Min Fu, Yucheng Zhang, and Yukun Zhou. 2016. A Comprehensive Study of the Past, Present, and Future of Data Deduplication. *Proc. IEEE* 104, 9 (2016), 1681–1710.
[16] Qirui Yang, Runyu Jin, and Ming Zhao. 2019. SmartDedup: Optimizing Deduplication for Resource-constrained Devices. In *ATC*. 633–646.
[17] Tianming Yang, Hong Jiang, Dan Feng, Zhongying Niu, Ke Zhou, and Yaping Wan. 2010. DEBAR: A scalable high-performance de-duplication storage system for backup and archiving. In *IPDPS*. 1–12.
[18] Ke Zhou, Shaofu Hu, Ping Huang, and Yuhong Zhao. 2017. LX-SSD: Enhancing the lifespan of NAND flash-based memory via recycling invalid pages. In *MSST*. 1–13.

FRM-CIM: Full-Digital Recursive MACComputing in Memory System Based on MRAM for Neural Network Applications

Jinkai Wang, Zekun Wang, Bojun Zhang, Zhengkun Gu, Youxiang Chen, Weisheng Zhao, Yue Zhang*

Fert Beijing Institute, MIIT Key Laboratory of Spintronics, School of Integrated Circuit Science and Engineering, Beihang University, Beijing 100191, China

Email: yz@buaa.edu.cn

Abstract—Computing in memory (CIM) realizes energy-efficient neural network algorithms by implementing highly parallel multiply-and-accumulate (MAC) operation. However, the MAC delay of CIM will sharply increase with the improvement of computing precision, which restricts its development. In this work, we propose a full-digital recursive MAC (FRM) operation based on spin-transfer-torque magnetic random access memory (STT-MRAM) CIM system to enable fast and energy-efficient image recognition application. First, the fast FRM scheme is proposed by utilizing the recursive operations of read and addition in segmented bit-line array, which effectively reduces the delay of MAC operations to 3.5ns and 4ns for 8-bit and 16-bit input and weight precision, respectively. Second, we design an image recognition system using FRM-CIM architecture as the processing element (PE), where the adaptive pruning method for layers is proposed to improve the compatibility of it with the neural network. By performing image recognition for the MNIST and CIFAR-10 datasets, results show that the throughput and energy efficiency of the FRM-CIM system are 58.51TOPS/mm^2 and 11.3 56.72 TOPS/W under 8 16 bit precision, which are improved by 4.3 times and 2.6 times compared with the state-of-the-art works. Finally, the recognition accuracy can reach 96.65% and 82.7% on MNIST and CIFAR-10, respectively.

Keywords Computing in memory (CIM), full-digital, recursive MAC, neural network, STT-MRAM.

I. INTRODUCTION

With the rapid development of artificial intelligence (AI), the data processed by neural network algorithms has increased exponentially. Von Neumann architecture is difficult to meet the performance requirements of AI because its structure of separated memory and computing results in a lot of energy and time spent on data transmission [1]. Computing in memory (CIM) eliminates above problem by integrating memory and computing functions. Meanwhile, CIM can perform highly parallel data processing, e.g., multiply-and-accumulate (MAC), by utilizing the characteristics of memory. Therefore, CIM is a promising approach to implement energy-efficient neural network processing with high performance.

As one of the cores of AI, neural network algorithm contains a large number of MAC operations. Utilizing the crossbar array structure of memory, the analog CIM architectures are proposed to efficiently implement MAC operations, as shown in Fig.1(a). The input data is converted from digital signals to analog signals of pulse or voltage through the digital-to-analog converters (DAC), which are loaded onto word line (WL) to activate bit cell stored the weight data. Each activated bit-cell will generate a logic current which represents the product property (i.e., I=VG)

DAC '24, June 23–27, 2024, San Francisco, CA, USA

ACM 979-8-4007-0601-1/24/06…

https://doi.org/10.1145/3649329.3655937

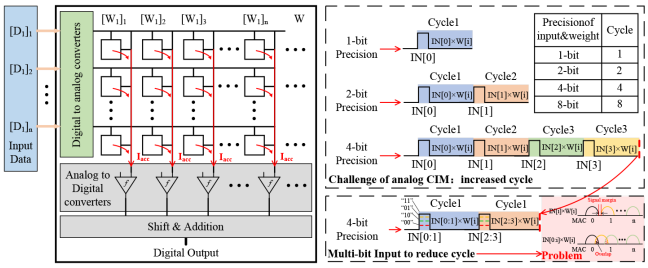


Fig. 1. (a) Analog CIM architecture. (b) Challenge of analog CIM.

from the input data on WL and the datum stored in the bit-cell. These logic currents generated by the bit-cells on a column converge on the bit-line (BL) to implement the accumulation operation. An analog-to-digital converter (ADC) converts the total current into a binary number on each column to realize single-bit MAC operations. Then single-bit MAC results on each column are weighted using shift and addition circuit to obtain the final result of the multi-bit MAC. Analog CIM architecture realizes high energy efficiency by exploiting the high parallelism of the columns (i.e., single-bit MAC operations can be performed on each column.). Nevertheless, as the computing precision increases, the number of cycles required to obtain the final MAC result also increases, resulting in higher delay that will seriously reduce computing energy efficiency, as shown in Fig.1(b). To solve this problem, [2] proposes the multi-bit input method to reduce computing cycles. However, multi-bit input method will cause the decrease in signal margin between adjacent MAC results, which greatly affects the recognition accuracy of the ADC. Although charge-domain [3] and time-domain [4] reading schemes effectively improve the signal margin, they will increase the delay of a single cycle. Therefore, reducing the delay of MAC operation with high energy efficiency and reliability has become the focus of CIM technology. Digital CIM architecture has high reliability by implementing Boolean logic since the number of activated bit cells in it is much smaller than analog CIM. At present, a variety of digital CIM schemes based on various memories have been proposed. Among them, spintronic memories are widely considered as a high potential candidate for digital CIM system designs, thanks to their intrinsic non volatility, low power consumption and high speed of read/write operations [5], [6]. However, the MAC operations of them are executed by scheduling Boolean logic, which increases the complexity of computing and reduces the energy efficiency.

To solve the above issues, combining the advantages of digital and analog CIM architecture, we propose a full-digital recursive MAC (FRM) CIM system by using segmented bit-line (BL) array structure in spin transfer torque magnetic random access memory (STT-MRAM), which greatly improves the delay and energy efficiency of MAC operation to increase the efficiency of performing image recognition. First, the FRM method is proposed by scheduling the read and addition operations, where a voltage following read cell

(VFRC) and segmented bit-line array are designed in order to recognize the datum of bit-cell with high parallelism and speed. Results show that the delay of MAC operation reduces to 3.5ns for 8-bit precision, which dropped by 1.7 times compared with [7]. Meanwhile, the MAC delay for16-bit and 32-bit precision are 4ns and 5.2ns respectively, which is the fastest among the reported works. Then, based on the FRM-CIM architecture, we construct an image recognition system. To further improve the compatibility of the FRM-CIM architecture with the neural network algorithms, we design the adaptive pruning method for each hidden layer. Finally, we evaluate the FRM-CIM system performance by performing image recognition for the MNIST and CIFAR-10 dataset. Results show that its throughput and energy efficiency are improved by 4.3 times and 2.6 times compared with state-of-the-art works. Meanwhile, the recognition accuracy can reach 96.65% and 82.7% on MNIST and CIFAR-10 dataset, respectively.

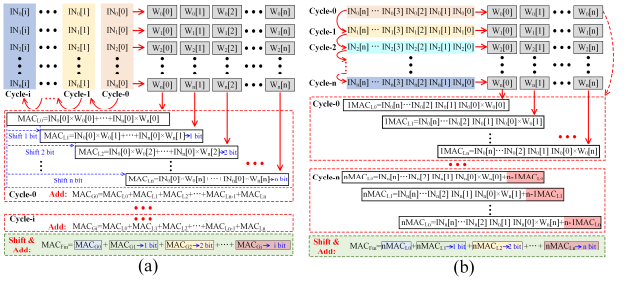
The rest of this article is organized as follows. Section II introduces the principle and structure of the proposed FRM-CIM architecture. Section III describes the designed image recognition system based on FRM-CIM architecture. Section IV presents the performance of the image recognition system. Conclusions are presented in Section V.

II. FRM-CIM Architecture

A. Principle of FRM scheme

Due to the recognition accuracy limitation of ADC circuit, the rang of MAC values distinguished on a column is limited. Therefore, the number of input data bits in a cycle is restricted, currently up to 3 bits, which results in an exponential increase in the number of execution cycles as the computing precision of the MAC operation increases, as shown in Fig. 2(a). For example, 8 cycles are usually required to complete the MAC operation of 8-bit input and weight. In each cycle, the signal-bit MAC operation is first implemented in each column of the memory array to achieve local MAC (MAC_{Ln}) result. Then the MAC_{Ln} result of each column will be weighted according to the weight and they are summed to obtain the global MAC (MAC_{Gi}) result. Finally, 8 MAC_{Gi} results obtained after 8 cycles will be weighted again according to the input and they are summed to achieve the final MAC (MAC_{Fin}) result. Although the number of cycles can be reduced by inputting multiple bits in one cycle, such as the cycle can be reduced to 4 cycles when 2 bits are input in one cycle, the rang of MAC values distinguished on a column is expanded resulting in the signal margin decline. Meanwhile, the shift operation for weighted values and addition are performed in each cycle of MAC operation caused by non-full precision input, thereby increasing delay and energy.

The proposed FRM scheme effectively solves the above problems through the full precision input, as shown in Fig. 2(b). In a cycle of FRM scheme, a full precision data is input into the CIM architecture to implement the multiplication operation with each bit of a weight data. They are added to the results obtained in the next cycle. For example, in first cycle-1, the first data IN_0 is input and multiplied by each bit of weight data W_0 on each column of memory array, i.e., $IN_0 \times W_0[0]$, $IN_0 \times W_0[1]$, …, $IN_0 \times W_0[n]$, which achieves n MAC ($1MAC_{Ln}$) result. Then, in second cycle-2, the second data IN_1 is input and multiplied by each bit of weight data W_1, which achieves n MAC ($2MAC_{Ln}$) result. At this time, the $2MAC_{Ln}$ result on each column are added to the $1MAC_{Ln}$ result from the cycle-1. In this way, the multiplication and addition operations are then performed in subsequent cycles, which is similar to recursive operations. In final cycle-n, the $iMAC_{Ln}$ result will be weighted according to the input and they are summed to achieve the final MAC (MAC_{Fin}) result. Obviously, the shift and addition due to the weighted operation are eliminated, which effectively reduces the complexity of MAC operations and improves computational efficiency in CIM architecture. Meanwhile, the number of cycles executed is only related to the number of accumulations and has nothing to do with the precision of the input data, which can greatly reduce the execution cycle of high precision MAC operations.



(a) (b)

Fig. 2. (a) Principle of the MAC operation in analog CIM. (b) Principle of the proposed FRM scheme.

B. FRM-CIM architecture

Based on the principle of FRM scheme, we designed the FRM-CIM architecture. Fig.3 shows the proposed structure of FRM-CIM scheme based on segmented BL technique, where a BL is divided into 16 segments and each segment has 8 bit-cells. To achieve connection in the memory model and disconnection in the computing model, BL and source-line (SL) of adjacent segments are connected through NMOS transistors controlled by the ENCIM signal. Moreover, each segment of BL is set the proposed voltage-following read cell (VFRC) to simultaneously read a cell performed computing in each segment. Meanwhile, the outputs of all VFRCs on a BL are connected to computing line (CL) through NMOS transistors controlled by the DEi signal. The register-accumulate circuit mainly consists of the rising edge-triggered D flip-flops (DFFs) and full adders (FAs). The proposed all-digital MAC operation includes two phases: (1) read the computing bit-cell, and (2) accumulate the input data.

Before the beginning of FRM scheme, the ENCIM is high voltage to turn on the NMOS transistors in the model switch (MS), which makes all segments on a BL connected.

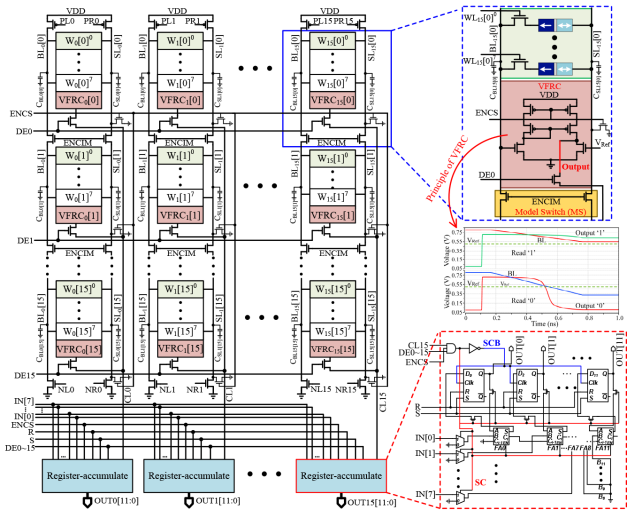


Fig.3. Architecture of the proposed FRM scheme.

Meanwhile, the BL is pre-charged to VDD by the PMOS transistor controlled by PLi signal. Then, ENCIM and EMCS are rise to VDD, causing each segment to enter the computing model independently. Note that the segmented BL technique also enables its parasitic capacitance to be equally divided, which means the parasitic capacitance of each segment ($C_{BLi[i]}$) is small. According to the characteristics of the proposed VFRC structure, this will greatly reduce the delay of the VFRC read operation. Besides, the read operation of the computing bit-cell in each segment on BL is carried out at the same time to decrease the access time of all-digital MAC operations. After that, the VFRCi[0] output of the first segment (i.e., 0 segment) on a BL is transferred to CLi by activated DE0 signal. For example, the VFRCi[0] output of '0' in Fig.4 (a) is transferred to CLi in Fig.4 (d). Note that the activation of DE0 requires a period of time for the register-accumulate circuit to perform addition and register operations, as shown in Fig.4 (b). In register-accumulate circuit, an AND logic is performed between CLi, DE0 and EMCS. If the CLi is '0' during the time that DEi is activated (i.e., the datum of weight is '0'), then SC signal is also '0', as shown in Fig.4 (e), which makes the FAs off and the DFFs in its original states, i.e, register-accumulate circuit does not work. That achieves weight-sparsity-aware energy-saving. On the contrary, if CLi is '1, for example, the VFRCi[1] output of '1' in Fig.4 (b) is transferred to CLi in Fig.4 (d) when DE1 is activated, then SC signal is '1' to enable the register-accumulate circuit work. At this time, the FA chain performs the addition operation for the input data (i.e., IN1[7:0]) and the data registered in DFFs. Because the SCB signal is the inverse of SC, SCB generates a rising edge when SC is a falling edge, as shown in Fig.4 (f), which is used as the Clk signal of rising edge-triggered DFF.

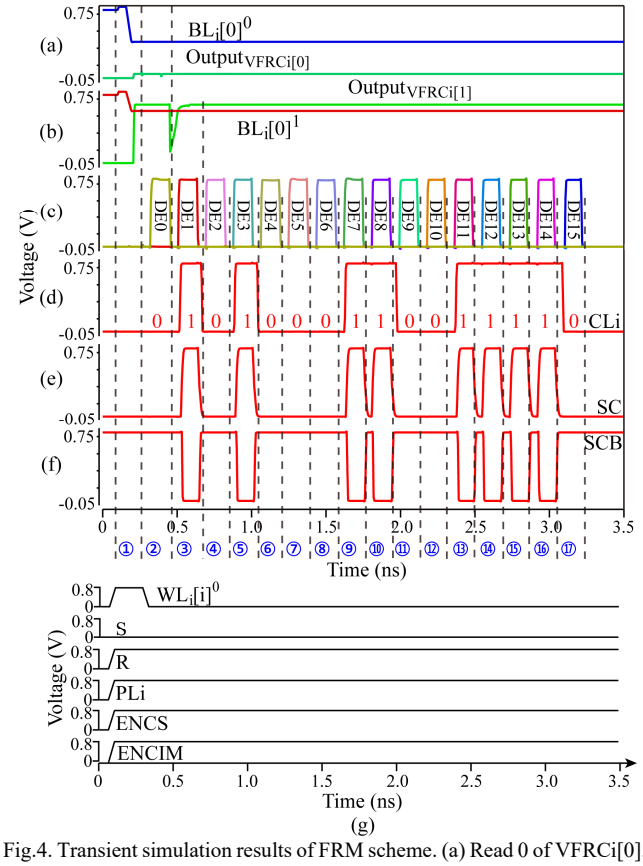


Fig.4. Transient simulation results of FRM scheme. (a) Read 0 of VFRCi[0]. (b) Read 0 of VFRCi[1]. (c) Waveform of DEi. (d) Waveform of CLi. (e) Waveform of SC. (f) Waveform of SCB. (g) Timing sequences of the proposed FRM scheme. ① Read each segment of BL in parallel. ②~⑰ Accumulation of input data.

On the other word, the DFFs will record the results generated by the FA chain when the DE0 activation ends. Therefore, the activation time of DEi is longer than the delay of the FA chain to ensure the accuracy of the computing. According to the above principle, the output of VFRC in each segment on a BL is transmitted to CLi sequentially to implement addition and register operations for input data in register-accumulate circuit, thereby realizing 16 accumulations without weight for 8-bit input data after activating DE0 to DE15. The main timing sequences of the proposed scheme are shown in Fig.4 (g). Finally, the 16 accumulations with weight for 8-bit input data can be achieved by the adder tree, where the OUT7 [11:0] is connected to the adder tree as the highest bit weight in the 8-bit weight MAC operation.

III. Image Recognition System Based on FRM-CIM Architecture

A. *Overall of image recognition system*

Based on FRM-CIM architecture, we further design an image recognition system to preform deep neural network (DNN). To efficiently provide input feature data, the buffer is assigned to the FRM-CIM architecture, thus constituting the process element (PE) in image recognition system. Note that the memory array of FRM-CIM architecture is divided into two parts through the NMOS transistors that are divided the WL into two segments, where each part has 8 columns, to realize the MAC operations of 8-bit and 16-bit precision in an architecture. Moreover, the 16 register-accumulate circuits are also divided into two parts according to the column connecting them. Besides, the proposed image recognition system also consists of the global data buffer (GDB), accumulator (ACC), data width pruner (DWP) and the finite state machine (FSM), as shown in Fig. 5. GDB stores the input feature from off-chip memory or input feature betweens layers and it allocates input feature data to the PE. 8 rows×32 columns PEs form a PE block. Moreover, the ACC circuit is integrated into each PE block to implement accumulation operations for the PE results. Then, the results of ACC module are transmitted to the DWP circuit, which will detect the maximum value of output feature and prune it to appropriate size. The principle of DWP is introduced in section B. Finally, the operations of the above circuits are controlled by FSM module. Besides, a predictor module is integrated into the image recognition system based on the FRM-CIM architecture to directly demonstrate the output feature.

Fig.6 shows the mapping scheme that the image data are input into the proposed image recognition system. Firstly, the

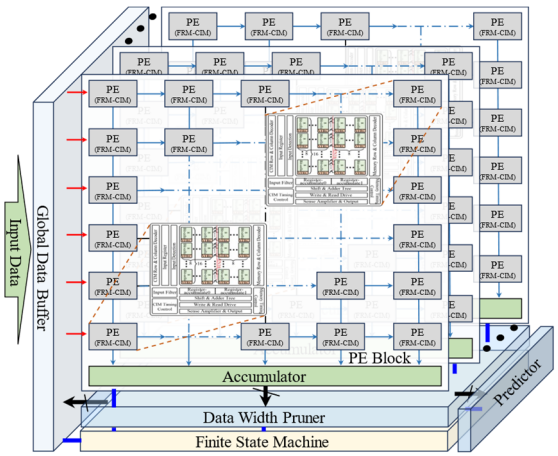


Fig.5. Image recognition system based on FRM-CIM architecture.

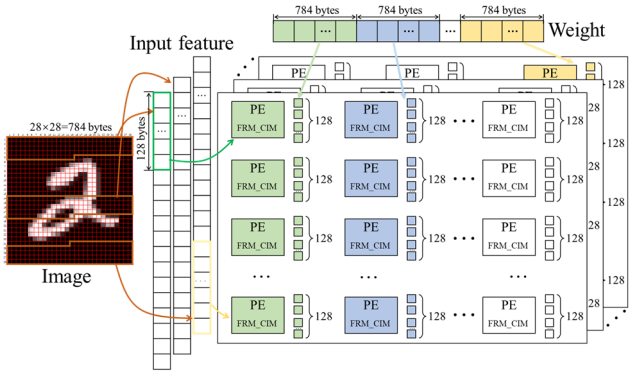


Fig.6. Mapping scheme of image recognition system based on FRM-CIM architecture.

weight matrix is expended into one long vector. The number of weights bytes that is the same as the number of input feature bytes is stored in a column of PEs. To improve the parallelism of computing, a column of PEs is set a channel and the input feature data is transmitted to it to implement DNN algorithm. Note that each PE can process 128 bytes of the input feature data. Therefore, a PE block can store no more than weights of 32 channels and the remaining weight bytes are stored in the next PE block.

B. Principle of data width pruner

In order to adapt to the different DNN algorithms and improve the compatibility of the FRM-CIM architecture with them, the proposed image recognition system will be repeatedly invoked to execute different layers of DNN. Therefore, the number of the input feature data bits need to be matched with the output data. However, the computation of each hidden layer results in an increase in the number of output data bits due to the data overflow. It is necessary to prune the output data to match the number of input features data bits.

However, the conventional pruning scheme that intercepts the fixed number of bits from the output data ignores the value range of the output data itself, which will cause the data to be clipped to zero or the data overflows. To solve this issue, we propose an adaptive pruning method that can determine the pruning position based on the distribution of the output data, as shown in Algorithm 1. In the proposed adaptive pruning method, a parameter reserve_bit is set and represents the number of signed bits reserved in pruning, ranging from 1 to 7. DWP circuit will first detect the index of the highest valid bit (HVB) from the output data generated by each channel (i.e., a column of PE). Note that the highest valid bit is the first bit opposite to the signed bit. The maximum index among them is selected and added to the reserve_bit to obtain the prune_bit. Then DWP will intercept the 8-bit data after the bit of prune_bit as the output data of it, which is stored in GDB as the input data of next hidden layer.

Algorithm 1: Prune for layers

```
1: Input: N: the number of input feature, W: the width of input feature data,
In_data: input feature data.
2: Output: Out_data: output feature data
3: Parameter: reserve_bit: bits reserved for signed bit
4: for i ← 1,2,3,…,N do
5:       for j ← W-1,W-2,W-3,…,1 then
6:            if In_data[i][j] ≠ In_data[i][W] then
7:                 HVB ← j
8:                 break
9:       if i = 1 then
10:           MAX_HVB ← HVB
11:      else then
12:           if HVB > MAX_HVB then
13:                MAX_HVB ← HVB
14: end
15: Prune_bit ← MAX_HVB + reserve_bit
16: for i ← 1,2,3,…,N do
17:      Out_data[i] ← In_data[i][Prune_bit:Prune_bit-8]
18: end
19: return Out_data
```

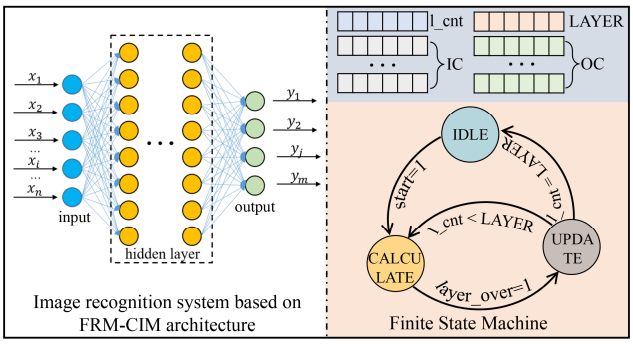


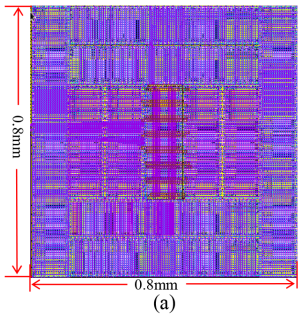
Fig.7. Working flow of the proposed image recognition system.

C. Working flow of image recognition system

Finite state machines (FSM) of image recognition system with a set of status signals is assigned to two-fixed point bits to represent IDLE state (“00”), COMPUTE state (“01”), and UPDATE state (“10”). IDLE state means the system is idle and is waiting for order. COMPUTE state means the system is computing a layer of DNN. UPDATE state means the system is updating parameters which are needed for computing a layer of DNN. The system receives signal “start” with FSM changing to CALCULATE state, and loads input feature. Input feature flows into PE array and becomes output feature, which is stored into GDB after pruning. When computing finish, the signal layer_over is set to 1 and FSM changes to UPDATE state. Inside of FSM, there a several register groups preserving parameters of current DNN layer, as shown in Fig. 7. l_cnt register group records the number of layers that has been computed. LAYER register group records the number of current DNN layers. IC register group records the input channel size of each layer. OC register group records the output channel size of each layer. These register groups are updated during UPDATE state. After updating, if the value stored in l_cnt register group is still smaller than that in LAYER, FSM turns to COMPUTE state. If the value stored in l_cnt register group equals to LAYER, output feature flows into predictor and the system outputs the predict label.

IV. Performance Evaluation and Analysis

Digital-analog hybrid simulations are implemented by using 14 nm CMOS process technology and the process design kit of MTJ. Fig. 8(a) shows the layout of the image recognition system based on FRM-CIM architecture, which can achieve the performance results closer to the chip. Fig. 8(b)



(a)

Parameter	Value
MTJ area	40 nm x 40 nm x π/4
Oxide barrier height of MTJ	0.85 nm
free layer height of MTJ	1.3 nm
Temperature	300 K
Nominal R_{MTJ} at R_L (R_H) of MTJ	3.98 KΩ (8.75 KΩ)
Critical switching current R_H - R_L of MTJ	28 μA
Critical switching current R_L - R_H of MTJ	72 μA
TMR	120%

(b)

Fig. 8 (a) Layout of the proposed image recognition system based on FRM-CIM architecture. (b) Key parameters of the MTJ device.

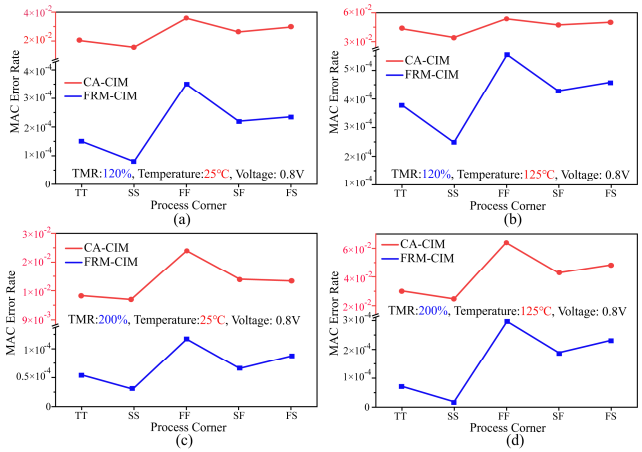


Fig.9. MER of the proposed FRM-CIM architecture under different process corner. (a) TMR of 120%, 25℃, 0.8V. (b) TMR of 120%, 125℃, 0.8V. (c) TMR of 200%, 25℃, 0.8V. (d) TMR of 200%, 125℃, 0.8V.

summarizes the key parameters of the MTJ device, which are dependent on physical models and experimental measurements [8].

As reliability is crucial for implementing computing, we first analyze the reliability of the proposed FRM-CIM architecture. In FRM-CIM architecture, the register-accumulate structure is composed of logic gates, which has high reliability. Therefore, the main factor that affects the reliability of the MAC operation comes from the VFRC circuit. Moreover, every MTJ is slightly different in its physical properties due to imperfections in the fabrication processes, as are every CMOS. We carry out the Monte Carlo simulations of 10^5 samples for the proposed FRM-CIM architecture to evaluate its reliability based on the parameter of MAC error rate (MER). Note that the process deviation of the MTJ resistance follows a Gaussian distribution with 5% variability [9]. Results show that the proposed FRM-CIM architecture reduces the error rate of MAC operation by two orders of magnitude compared with the conventional analog CIM (CA-CIM) architecture, as shown in Fig.9. Besides, increasing the TMR can increase the difference between high resistance state and low resistance state of MTJ, which improves the read operation accuracy of the proposed VFRC circuit, thereby further reducing the MER value. Obviously, the MAC operation accuracy of the FRM-CIM architecture is comparable to that of the digital circuit.

Fig.10 shows the delay and energy of the FRM-CIM architecture when performing 8-bit and 16-bit MAC operation under different supply voltage, where the MAC operation is divided into four parts: row and column decoding operation, read operation of VFRC, register-accumulate operation as well as shift and addition (Shift & Add) operation. Obviously, the delay and energy of register-accumulate operation are the largest, which is caused by the following two reasons: 1) the

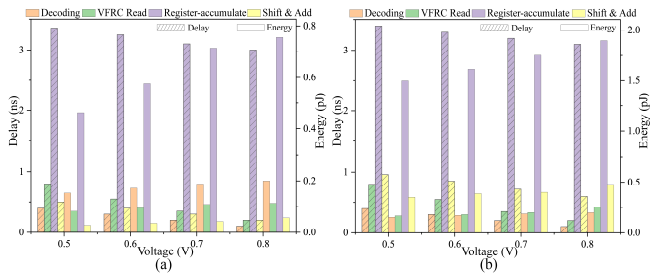


Fig.10. Delay and energy of FRM-CIM architecture when performing MAC operation under different supply voltage. (a) Input and weight are both 8-bit precision. (b) Input and weight are both 16-bit precision.

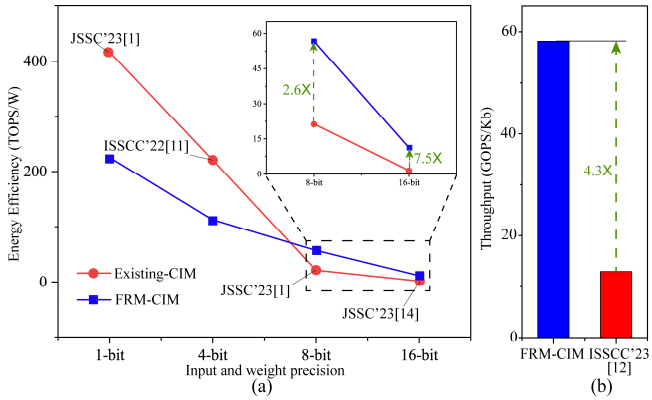
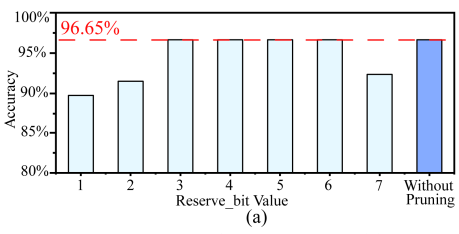


Fig.11. Performance of the FRM-CIM architecture performing MAC operation. (a) Energy efficiency under different input and weight precision. (b) Normalized to throughput per Kb

register-accumulate circuit contains a large number of DFFs and FAs to implement MAC operations. 2) The DFFs needed to be powered on all the time during the computing to store the results. Therefore, an effective method to reduce energy consumption in FRM-CIM architecture is to use the low-power DFF proposed by various literatures [10]. Besides, it can also be observed that the delay and energy of decoding operation in 8-bit MAC operation are almost the same as those in 16-bit, which benefits from the full precision data input to enable the same number of decoding operations to be performed when the number of accumulations is the same. Meanwhile, Fig.9 demonstrates that the difference in MAC operation delay under different input and weight precisions mainly comes from the Shift & Add operations, i.e., the delay of performing the weighted addition for the single-bit MAC results generated by each column. Therefore, the proposed FRM-CIM architecture eliminates the correlation between delay and precision of MAC operation in the analog CIM.

Fig.11 shows the energy efficiency and throughput of the FRM-CIM architecture performing MAC operation. For 1-bit and 4-bit input and weight precision, the energy efficiency of the FRM-CIM architecture is lower than the existing CIM architectures. The reason for this problem is that the FRM-CIM architecture needs to perform a large number of addition and register operations by using logic gates (i.e., register-accumulate circuit). However, with the improvement of input and weight precision, the number of cycles in analog CIM architecture increases exponentially, resulting in its energy being much greater than that of the register-accumulate circuit. Therefore, it can be observed in Fig.11 (a) that the energy efficiency FRM-CIM architecture at 8-bit and 16-bit is increased by 2.6 times and 7.5 times, respectively, compared with the state-of-the-art works. Besides, compared with the [12], the throughput of FRM-CIM architecture is improved by 4.3 times due to its smaller delay, as shown in Fig.11 (b).

In the designed image recognition system based on the FRM-MAC architecture, an adaptive pruning method is



	MNIST	CIFAR -10
Accuracy	96.65%	82.7%
Execution Time	12μs	42μs
Power	38mW	112mW

(b)

Fig.12. Performance of the image recognition system based on the FRM-MAC architecture (a) Accuracy for MNIST dataset. (b) Accuracy, time and power for executing DNN to identify MNIST and CIFAR-10 datasets.

TABLE II COMPARISON WITH PREVIOUS WORKS

		This work		ISSCC'23[13]	JSSC'23[1]	ISSCC'23[7]	ISSCC'23[12]	JSSC'23[14]		ISSCC'23[15]
CMOS Technology		**14nm**		22nm	22nm	28nm	4nm	28nm		22nm
Memory Type		**STT-MRAM**		STT-MRAM	ReRAM	SRAM	SRAM	SRAM		SRAM
Capacity		**2Mb**		8Mb	8Mb	4Kb	54Kb	16Kb		13Kb
Supply Voltage (V)		**0.5~0.8**		0.8	0.8	0.6~0.9	0.32~1	0.7/0.8		0.72~0.82
CIM mode		**Full-Digital**		Analog	Time-Domain	Analog	Digital	Digital		Analog
Bit Precision (bit)	Input	**8**	**16**	8	8	8	8	8	16	8
	Weight	**8**	**16**	8	8	8	8	8	16	8
	Output	**24**	**36**	26	19	20	24	NA	NA	24
Access Time (ns)[1]		**3.5**	**4**	10.56	14.4	6	6	180	340	NA
Throughput (TOPS/mm^2)		**58.51**	**51.2**	0.05	0.387	1.125	13.7	0.27	0.068	1.4
Energy Efficiency (TOPS/W)		**56.72**	**11.3**	46.4[2]	21.6	33.44	41.3	4.55	1.5	21.38

[1]Delay of the MAC operation; [2]50% input sparsity.

adapted to crop the output data of the hidden layer to a low precision data determined by the parameter reserve_bit value, which leads to the decrease in image recognition accuracy. If the reserve_bit value is small, most bits of data are reserved, which causes the data overflow during subsequent computations. Meanwhile, if the reserve_bit value is big, only little bits of data are reserved, which causes data information loss. Fig.12 provides a detailed analysis of the reserve_bit value on image recognition accuracy for MNIST dataset. It can be observed that the accuracy loss is large when the reserve_bit value is 1 and 7. When it is 3, 4, 5 and 6, there is no accuracy loss compared with the without pruning. Table II compares the proposed FRM-CIM architecture with state-of-the-art CIM architectures published in the recent years. Compared with [13] and [1] that also build based on the non-volatile memory, the access time of performing MAC operation in the FRM-CIM architecture has been greatly decreased. Besides, the energy efficiency of 16-bit input and weight MAC operation in FRM-CIM architecture is up to 11.3 TOPS/W, which is a great advantage compared with SRAM-based CIM [14] known for its high energy efficiency.

V. CONCLUSION

This work proposes an FRM-CIM architecture to perform fast and energy-efficient execution of the MAC operation by utilizing segmented BL array in STT-MRAM. First, the FRM operation based on recursive execution of addition and register operations is presented to realize full precision input of data, which efficiency eliminates the correlation between input precision and delay, thereby greatly reducing the delay of high precision MAC operation in the CIM architecture. Meanwhile, based on the FRM-CIM architecture, we further construct an image recognition system to perform image recognition for the MNIST and CIFAR-10 datasets by using DNN algorithm, where an adaptive pruning method for each hidden layer is proposed to improve the compatibility of the RM-CIM architecture with DNN. Finally, results in layout level show that the FRM-CIM architecture achieves 56.72 TOPS/W and 11.3 TOPS/W for 8-bit and 16-bit MAC operations, respectively, outperforming existing CIM architectures. In summary, this work has significance for further research on MRAM to realize high throughput and energy-efficient neural network platform.

ACKNOWLEDGEMENT

This work was supported in part by the National Natural Science Foundation of China under Grant 61971024, 62122008, and 51901008, and in part by the Project funded by China Postdoctoral Science Foundation 2022M720344.

REFERENCES

[1] J. M. Hung et al., "8-b Precision 8-Mb ReRAM Compute-in-Memory Macro Using Direct-Current-Free Time-Domain Readout Scheme for AI Edge Devices," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 1, pp. 303-315, Jan. 2023.

[2] C. X. Xue et al., "A 22nm 4Mb 8b-Precision ReRAM Computing-in-Memory Macro with 11.91 to 195.7TOPS/W for Tiny AI Edge Devices," *IEEE International Solid-State Circuits Conference*, San Francisco, CA, USA, 2021.

[3] H. Wang et al., "A Charge Domain SRAM Compute-in-Memory Macro With C-2C Ladder-Based 8-Bit MAC Unit in 22-nm FinFET Process for Edge Inference," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 4, pp. 1037-1050, April 2023.

[4] Y. Zhang et al., "Time-Domain Computing in Memory Using Spintronics for Energy-Efficient Convolutional Neural Network," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 68, no. 3, pp. 1193-1205, March 2021.

[5] S. Jung et al. "A crossbar array of magnetoresistive memory devices for in-memory computing," *Nature*, vol. 601, pp. 211-216, Jan. 2022.

[6] J. Wang et al., "TAM: A Computing in Memory based on Tandem Array within STT-MRAM for Energy-Efficient Analog MAC Operation," *Design, Automation & Test in Europe Conference & Exhibition*, Antwerp, Belgium, 2023

[7] B. Wang et al., "A 28nm Horizontal-Weight-Shift and Vertical-feature-Shift-Based Separate-WL 6T-SRAM Computation-in-Memory Unit-Macro for Edge Depthwise Neural-Networks," *IEEE International Solid-State Circuits Conference*, San Francisco, CA, USA, 2023.

[8] Y. Zhang et al., "Compact modeling of perpendicular-anisotropy CoFeB/MgO magnetic tunnel junctions," *IEEE Transactions on Electron Devices*, vol. 59, no. 3, pp. 819–826, Mar. 2012.

[9] J. K. Wang et al., "A self-matching complementary-reference sensing scheme for high-speed and reliable toggle spin torque MRAM," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 67, no. 12, pp. 4247–4258, Dec. 2020.

[10] G. Scotti et al., "Design of Low-Voltage High-Speed CML D-Latches in Nanometer CMOS Technologies," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 25, no. 12, pp. 3509-3520, Dec. 2017.

[11] H. Fujiwara et al., "A 5-nm 254-TOPS/W 221-TOPS/mm2 Fully-Digital Computing-in-Memory Macro Supporting Wide-Range Dynamic-Voltage-Frequency Scaling and Simultaneous MAC and Write Operations," *IEEE International Solid-State Circuits Conference, San Francisco*, CA, USA, 2022.

[12] H. Mori et al., "A 4nm 6163-TOPS/W/b 4790-TOPS/mm^2/b SRAM Based Digital-Computing-in-Memory Macro Supporting Bit-Width Flexibility and Simultaneous MAC and Weight Update," *IEEE International Solid-State Circuits Conference, San Francisco*, CA, USA, 2023.

[13] Y. C. Chiu et al., "A 22nm 8Mb STT-MRAM Near-Memory-Computing Macro with 8b-Precision and 46.4-160.1TOPS/W for Edge-AI Devices," *IEEE International Solid-State Circuits Conference*, San Francisco, CA, USA, 2023.

[14] C. Y. Yao, T. Y. Wu, H. C. Liang, Y. K. Chen and T. T. Liu, "A Fully Bit-Flexible Computation in Memory Macro Using Multi-Functional Computing Bit Cell and Embedded Input Sparsity Sensing," IEEE Journal of Solid-State Circuits, vol. 58, no. 5, pp. 1487-1495, May 2023.

[15] P. Chen et al., "A 22nm Delta-Sigma Computing-In-Memory ($\Delta\sum$CIM) SRAM Macro with Near-Zero-Mean Outputs and LSB-First ADCs Achieving 21.38TOPS/W for 8b-MAC Edge AI Processing," I*EEE International Solid-State Circuits Conference*, San Francisco, CA, USA, 2023.